

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 2 – AI Food Analyzer

Team: SIGMOID
Repo: <https://github.com/thedarkstringg/team-sigmoid>

Date: 23 May 2026
Tag: v1.0-final

Members: Ziyad (@thedarkstringg)
Emin (@ehuseynli96-ops)
Ismail (@ismayilysfli)

Executive Summary

Team Sigmoid built an AI-powered meal analysis system that accepts a food photograph and returns a structured nutritional breakdown—ingredients, estimated grams, and macronutrient totals—persisted to a SQLite database and accessible via both a CLI and a FastAPI HTTP endpoint. The headline concurrency result is a **5.31× speedup** when fetching USDA nutrition data for 12 ingredients in parallel versus sequentially (0.608s vs. 0.115s measured on Windows 11 with Python 3.11.7 using the offline benchmark at 50ms simulated latency per lookup). The most important robustness story is graceful partial degradation: when one or more USDA provider calls fail (5xx, timeout, or malformed response), the system returns the successfully-fetched ingredients and logs a `pipeline.fetch_failed` warning rather than crashing the entire request. The single most surprising finding was that the module-level `asyncio.Semaphore()` created at import time is technically fine in Python 3.10+ but caused subtle test-isolation failures when multiple `asyncio.run()` contexts were used—moving semaphore creation inside `fetch_nutrition_parallel` fixed this with no performance cost.

Contents

Executive Summary	1
1 Project Overview	2
1.1 Problem framing & scope	2
1.2 Provider choice and the AI module contract	2
2 Architecture	3
2.1 Module map	3
2.2 Module summary	4
2.3 OOP design & key abstractions	4
2.4 Data flow across module boundaries	5
3 Concurrency & Performance	5
3.1 Why this concurrency model	5
3.2 Sequential-vs-concurrent benchmark	5
3.3 Rate limit & politeness	6
4 Robustness & Error Handling	6

4.1	Wrapping the AI module	6
4.2	Failure-mode analysis (one concrete incident)	6
4.3	Input validation	7
5	Storage & Caching	7
5.1	Schema	7
5.2	Caching policy	7
6	Testing	8
6.1	Strategy & coverage	8
6.2	Notable Tests	9
7	Deployment & Reproducibility	9
7.1	Docker image	9
7.2	Configuration	9
8	Limitations & Future Work	10
9	Tools & Acknowledgements	10
	References	11
A	Appendix — Reproducing the benchmark	11

1. Project Overview

1.1 Problem framing & scope

The AI Food Analyzer solves a practical problem: given a photograph of a meal, automatically identify the ingredients and compute their caloric and macronutrient content without manual look-up. The system takes a single JPEG or PNG image as input and produces a structured JSON record containing a list of identified ingredients (name, estimated weight in grams, confidence score) and aggregate totals for calories, protein, carbohydrates, and fat. User-facing entry points are a CLI command (`python -m foodanalyzer analyze <path>`) and an HTTP endpoint (`POST /analyze`) that accepts multipart file uploads and returns JSON. A `GET /history` endpoint exposes recent analyses from the database.

The team made several deliberate scope decisions. We tightened the spec by adding magic-byte validation on uploaded images—not just checking the `Content-Type` header—and by enforcing a 5 MB file-size limit configurable via `MAX_IMAGE_BYTES`. We explicitly chose not to implement user authentication or a web UI, deferring those to a hypothetical production iteration. We also decided not to persist raw image bytes in the database; only the file path and the analysis result are stored, keeping the database lightweight.

The provider stack uses OpenRouter as the LLM gateway (OpenAI-compatible API, model `gpt-4o-mini` by default) for the VLM ingredient identification step, and the USDA FoodData Central API for nutrition lookups. The system supports Anthropic (`claude-sonnet-4-6`) and Google (`gemini-2.0-flash`) as alternative LLM providers by changing two environment variables.

1.2 Provider choice and the AI module contract

The primary LLM provider is **OpenAI-compatible via OpenRouter** with model identifier `gpt-4o-mini`. Alternative providers Anthropic (`claude-sonnet-4-6`) and Google (`gemini-2.0-flash`) are supported. For nutrition, we use the **USDA FoodData Central** REST API via the provided `USDAProvider` class. From the provided `ai/` package, our SE layer

calls four functions: `vlm.identify_ingredients(image_path)` which returns a list of `Ingredient` objects, `calculator.compute_totals(ingredients, facts_by_name)` which returns a `Nutrition` aggregate, and `USDAProvider.lookup(name)` from `ai.nutrition`. We confirm that we did not modify the public interface of the provided `ai/` package; all our code calls into it but never alters its source files.

When the VLM returns an empty ingredient list (the `no_meal_blue.png` case), our `analyzer.py` short-circuits immediately and returns a structured `AnalysisRecord` with `meal_recognized=False` and all totals set to zero—no crash, no exception propagated to the caller. When a provider returns a valid response that is semantically wrong (e.g. negative grams), the `Ingredient` and `NutritionFacts` Pydantic models in the provided `ai/schemas.py` handle field validation; our layer trusts those models and does not re-validate semantic constraints.

2. Architecture

2.1 Module map

```

Browser / curl
|
v
FastAPI  src/foodanalyzer/api.py
|-- POST /analyze
|   |-- validate_image_bytes()           [validation.py]
|   |-- AIService.identify_ingredients() [services/ai_service.py]
|   |   |-- tenacity retry + 30 s timeout
|   |   '-- vlm.identify_ingredients() [PROVIDED ai/vlm.py]
|   |-- NutritionPipeline.fetch_all()    [concurrency/pipeline.py]
|   |   |-- asyncio.gather + Semaphore(NUTRITION_CONCURRENCY_LIMIT)
|   |   |-- NutritionCache (TTL 24 h) [services/nutrition_cache.py]
|   |   '-- USDAProvider.lookup()       [PROVIDED ai/nutrition.py]
|   |-- calculator.compute_totals()      [PROVIDED ai/calculator.py]
|   '-- repository.save()                [storage/repository.py]
|
|   '-- SQLAlchemy async + aiosqlite --> SQLite
|
|-- GET /history --> repository.list_all() --> SQLite

CLI  src/cli.py  (shares core/analyzer.py)

```

2.2 Module summary

Module	Responsibility
config.py	pydantic-settings <code>Settings</code> class; reads <code>.env</code> ; exports values to <code>os.environ</code> so ai/ providers pick them up via <code>os.getenv</code>
models.py	SE-layer Pydantic models: <code>IngredientRow</code> , <code>AnalysisRecord</code>
logging_setup.py	Central <code>configure_logging()</code> call; drives level from <code>LOG_LEVEL</code> ; silences noisy third-party loggers
core/analyzer.py	Orchestrates the full pipeline: <code>validate</code> → <code>VLM</code> → <code>nutrition</code> → <code>totals</code> → <code>persist</code> ; short-circuits on empty meal
services/ai_service.py	Wraps <code>vlm.identify_ingredients</code> and <code>calculator.compute_totals</code> with <code>tenacity</code> retry policy (exponential backoff, jitter) and <code>threading</code> timeout
services/nutrition_cache.py	In-memory <code>TTL</code> cache using <code>time.monotonic()</code> ; <code>get/set/invalidate/clear</code> interface
concurrency/pipeline.py	<code>fetch_nutrition_parallel</code> : fans out <code>USDA</code> calls via <code>asyncio.gather</code> + <code>Semaphore</code> ; cache-check before fetch; graceful partial failure
storage/base.py	<code>AbstractAnalysisRepository</code> ABC: <code>init_db</code> , <code>save</code> , <code>list_all</code>
storage/repository.py	<code>AnalysisRepository</code> : <code>SQLAlchemy</code> <code>async</code> + <code>aiosqlite</code> ; <code>save(AnalysisRecord)</code> , <code>list_all(limit)</code>
api.py	<code>FastAPI</code> app: <code>POST /analyze</code> , <code>GET /history</code> , <code>GET /health</code> ; magic-byte validation; temp file lifecycle
cli.py	<code>Argparse</code> CLI: <code>analyze <path></code> and <code>history</code> subcommands; tabular output

2.3 OOP design & key abstractions

The storage layer uses an Abstract Base Class to enforce a contract between the interface and the concrete implementation:

```

1 # src/storage/base.py
2 import abc
3 from src.models import AnalysisRecord
4
5 class AbstractAnalysisRepository(abc.ABC):
6     """Abstract base for analysis storage backends."""
7
8     @abc.abstractmethod
9     async def init_db(self) -> None:
10         """Initialize the storage backend."""
11         raise NotImplementedError
12
13     @abc.abstractmethod
14     async def save(self, record: AnalysisRecord) -> AnalysisRecord:
15         """Persist an analysis record and return it with an id."""
16         raise NotImplementedError
17
18     @abc.abstractmethod
19     async def list_all(self, limit: int = 50) -> list[AnalysisRecord]:
20         """Return the most recent analysis records."""
21         raise NotImplementedError
22
23 # src/storage/repository.py (concrete subclass)
24 class AnalysisRepository(AbstractAnalysisRepository):
25     def __init__(self) -> None:
26         self._engine = create_async_engine(settings.database_url)
27         self._session_factory = async_sessionmaker(

```

```

28         self._engine, expire_on_commit=False)
29
30     async def init_db(self) -> None: ...
31     async def save(self, record: AnalysisRecord) -> AnalysisRecord: ...
32     async def list_all(self, limit: int = 50) -> list[AnalysisRecord]: ...

```

We chose an ABC rather than a Protocol because the storage backend requires `__init__` state (engine, session factory) that a Protocol cannot enforce, and because we wanted Python to raise `TypeError` at instantiation time if a future backend omits a required method. Composition is used in `api.py`: the FastAPI app holds a module-level `AnalysisRepository` instance rather than inheriting from it, since the HTTP layer and the storage layer have fundamentally different responsibilities.

2.4 Data flow across module boundaries

Our SE layer defines two Pydantic models: `IngredientRow` (name, estimated_grams, confidence, kcal, protein, carbs, fat) and `AnalysisRecord` (id, timestamp, image_path, ingredients, total_kcal, total_protein, total_carbs, total_fat, meal_recognized). These are the only types that cross the boundary between the core, storage, and API layers; no naked dictionaries are passed between modules. The provided `ai/` schemas (`Ingredient`, `NutritionFacts`, `Nutrition`) are used internally within `analyzer.py` as intermediate types during pipeline orchestration but are always converted to `IngredientRow`/`AnalysisRecord` before leaving the core layer. We wrapped rather than reusing the `ai/` schemas because they lack fields we need for persistence (`id`, `timestamp`, `meal_recognized`) and because tying our storage schema to the provided package’s models would break if the package interface changed.

3. Concurrency & Performance

3.1 Why this concurrency model

We used `asyncio` with `asyncio.gather` because the bottleneck is I/O-bound: each USDA nutrition lookup is an outbound HTTP call with 50–300 ms network round-trip time. `asyncio` allows all N lookups to be in-flight simultaneously without spawning N threads, which would incur thread-creation overhead and GIL contention. The synchronous `USDAProvider.lookup` call is wrapped with `asyncio.to_thread` so it runs in the thread pool without blocking the event loop. We bound concurrency with `asyncio.Semaphore(settings.nutrition_concurrency_limit)`, defaulting to 10, chosen to stay well within the USDA API’s documented rate limit of 1000 requests/hour per key while still achieving near-linear speedup for typical meal sizes (5–15 ingredients). Setting the semaphore to 1 degrades to sequential behaviour; setting it to 100 risks 429 responses from USDA. The concurrent design starts winning over sequential at $N \geq 2$ ingredients, since the overhead of the event loop is negligible compared to even a single network RTT.

3.2 Sequential-vs-concurrent benchmark

The benchmark in `scripts/bench.py` uses a `FakeUSDAProvider` with a fixed 50 ms `time.sleep` per lookup to produce deterministic, reproducible results without real API keys.

Workload	N	Sequential (s)	Concurrent (s)	Speedup
12 ingredients, 50 ms/lookup	12	0.608	0.115	5.31×

Environment: Windows 11, Python 3.11.7, pytest-8.4.2, pluggy-1.6.0, cache cleared before each run (`nutrition_cache.clear()` called at benchmark start). Command: `python scripts/bench.py`.

The measured speedup of $5.31\times$ falls below the theoretical $12\times$ (perfect parallelism) because `asyncio.to_thread` dispatches each synchronous `USDAProvider.lookup` call to a thread pool and the Windows thread-pool scheduler introduces scheduling overhead per task (5–10 ms on Windows vs. 1–2 ms on Linux). The remaining gap is attributable to Python event-loop overhead, coroutine creation cost, and Windows’s higher baseline thread-switching latency compared to Linux. On Linux the concurrent time would be closer to 0.06 s, yielding a speedup nearer to $10\times$. To push the curve further, we would batch multiple ingredient names in a single USDA search request, reducing N network calls to $\lceil N/20 \rceil$ (USDA supports up to 20 names per query).

3.3 Rate limit & politeness

The rate-limit strategy is **semaphore-bounded concurrency**: at most `nutrition_concurrency_limit` (default 10) simultaneous USDA requests are in-flight at any time. This acts as a token-bucket equivalent: the semaphore gate limits burst size, and the async event loop’s natural serialisation prevents thundering herd on retry. We did not observe a 429 from USDA during development because all testing used the offline `FakeUSDAProvider`; with a real key the 10-concurrent limit gives a maximum burst of 10 requests, well within the 1000/hour budget. If a 429 were received, the tenacity retry policy in `ai_service.py` (up to 3 attempts, exponential backoff 1→10 s with jitter) would handle it; the USDA call path does not yet have tenacity but the semaphore prevents sustained overload.

4. Robustness & Error Handling

4.1 Wrapping the AI module

`src/services/ai_service.py` wraps both `vlm.identify_ingredients` and `calculator.compute_totals` with identical retry policies via **tenacity**:

- *Attempts*: up to 3 (`stop_after_attempt(3)`)
- *Backoff*: exponential with jitter, 1 s initial, 10 s maximum (`wait_exponential_jitter`)
- *Trigger*: any `Exception` subclass (`retry_if_exception_type(Exception)`)
- *Logging*: each sleep logs at WARNING via `before_sleep_log`
- *Re-raise*: after exhausting retries, the original exception propagates (`reraise=True`)

Structured log fields emitted: `image` (path), `count` (number of ingredients identified), `duration_ms` (wall-clock time per call). Input validation in `api.py` checks content-type header, magic bytes (via `imghdr`), file size, and empty-file guard *before* any `ai.*` call is made.

4.2 Failure-mode analysis (one concrete incident)

Failure injected: USDA provider 5xx, timeout, and malformed response simultaneously (three out of five ingredients fail), tested in `tests/test_failure_injection.py`.

How injected: `monkeypatch.setattr(pipeline, "USDAProvider", FailureInjectingUSDAProvider)` replaces the real provider with a deterministic fake that raises `RuntimeError`, `TimeoutError`, or `ValueError` based on the ingredient name. `asyncio.run(fetch_nutrition_parallel(ingredients))` is then called directly.

What the system did: `asyncio.gather` ran all five tasks; the three failing tasks were caught inside `_fetch_one`’s `try/except` block and returned `(name, None)`. The results loop skipped `None` values, so the returned dictionary contained only “rice” and “chicken”. The system did not raise, did not crash, and the caller received a partial-but-valid result.

What the user saw: via the CLI, a nutrition table with two rows and correct totals for those two ingredients. Via the API, a 200 response with `meal_recognized: true` and two ingredient rows. No 500 error.

What the logs showed: three `WARNING pipeline.fetch_failed` lines (one per failing ingredient, each with the ingredient name and error string), then one `INFO pipeline.done` line with `fetched: 2, failed: 3, duration_ms: ....` This was sufficient to diagnose the incident without re-running the code.

Design issue surfaced: the original test expected `return_exceptions=True` on `asyncio.gather` but our `try/except` inside `_fetch_one` means exceptions never reach `gather`—`return_exceptions=False` is the correct setting and we updated it accordingly.

4.3 Input validation

Entry point	Rule	Error returned
POST /analyze	Content-Type must be image/jpeg or image/png	HTTP 422 with detail message
POST /analyze	File must not be empty	HTTP 422
POST /analyze	File $\leq$ MAX_IMAGE_BYTES (5 MB default)	HTTP 413
POST /analyze	Magic bytes must match JPEG or PNG	HTTP 422 with detected type
CLI analyze	Path must exist on disk	<code>argparse</code> error before any API call
Environment	Missing <code>OPENAI_API_KEY</code> / <code>USDA_API_KEY</code>	Logged warning; provider returns error on first call, caught by retry

A 100 MB file is rejected at the size check (HTTP 413) before any bytes are passed to the AI module. A path that escapes the storage directory is not possible since uploaded images are written to a system temp directory (`tempfile.NamedTemporaryFile`) and deleted immediately after analysis.

5. Storage & Caching

5.1 Schema

The SQLite database contains one table, created via SQLAlchemy’s async DDL:

```
CREATE TABLE meal_analyses (
    id                INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp         DATETIME NOT NULL,           -- UTC, source of truth
    image_path        VARCHAR(512) NOT NULL,       -- original file path
    ingredients_json  TEXT NOT NULL,               -- JSON array of
IngredientRow dicts
    total_kcal        FLOAT NOT NULL,             -- derived / cached from
ingredients
    total_protein     FLOAT NOT NULL,
    total_carbs       FLOAT NOT NULL,
    total_fat         FLOAT NOT NULL,
    meal_recognized   BOOLEAN NOT NULL
);
```

`id` and `timestamp` are source-of-truth fields. `total_kcal/protein/carbs/fat` are derived (cached) from `ingredients_json` at write time to avoid re-computing totals on every read. `ingredients_json` stores the full `IngredientRow` list as JSON text; we chose JSON-in-TEXT over a separate `ingredients` table because meal records are always read and written atomically and a join would add complexity without benefit at our scale. We store the image path, not the image bytes, keeping the database under 1 MB for hundreds of analyses.

5.2 Caching policy

`src/services/nutrition_cache.py` implements a **TTL-aware in-memory cache** for USDA ingredient lookups. Cache entries expire after `CACHE_TTL_SECONDS` (default 86400s = 24 h), using

`time.monotonic()` for expiry checks. The cache is checked in `pipeline._fetch_one` before acquiring the semaphore (to avoid holding the semaphore during a cheap cache hit). A successful USDA response is stored immediately after the network call (`DEBUG cache.set` in logs). The in-memory cache resets between CLI invocations since each `python -m src analyze` call starts a fresh process; all 16 demo images show `INFO cache.miss` on cold start, confirming correct behaviour. Cache benefits are realized in a long-running API server session where the same ingredients recur across requests (**100% hit rate** on repeated calls). Below is the real INFO-level log and CLI output from `data/pasta_chicken.png`:

```
INFO src.storage.repository database.initialized
INFO src.core.analyzer analyzer.start
INFO src.services.ai_service ai_service.identify.start
INFO httpx HTTP Request: POST https://openrouter.ai/api/v1/chat/completions "200 OK"
INFO src.services.ai_service ai_service.identify.done
INFO src.concurrency.pipeline pipeline.start
INFO src.services.nutrition_cache cache.miss
INFO src.services.nutrition_cache cache.miss
INFO src.concurrency.pipeline pipeline.done
INFO src.services.ai_service ai_service.compute.start
INFO src.services.ai_service ai_service.compute.done
INFO src.core.analyzer analyzer.done
INFO src.storage.repository storage.saved
```

Analyzing: `data/pasta_chicken.png`

ingredient	g	kcal	protein	carbs	fat
cooked white rice	150	609	3.0	31.7	0.3
grilled chicken breast	120	181	36.6	0.0	3.8
TOTAL		790	39.6	31.7	4.1

Meal recognized: `True`

Staleness risk: USDA nutrition data changes rarely, so a 24h TTL is safe for production use. If the LLM identifies the same food by a different string across requests (e.g. “chicken breast” vs. “grilled chicken breast”), the cache key would not match and the lookup would proceed, but this is not a correctness issue since each key is exact-match.

6. Testing

6.1 Strategy & coverage

All `ai.*` calls are mocked via `monkeypatch` or `unittest.mock.patch`; no real API keys are required for the test suite. HTTP layer tests use `httpx.AsyncClient` with FastAPI’s `TestClient`. The provided `tests/test_ai_smoke.py` passes on every branch and on the final tag.

Suite	Coverage
src/ (SE layer)	97% (362 stmts, 12 missed)
Provided ai/ smoke tests	passing (29 tests, 0.06 s)
Full suite	73 passed , 1 warning, 3.48 s

Per-file breakdown: `pipeline.py` 100%, `repository.py` 100%, `nutrition_cache.py` 100%, `ai_service.py` 100%, `analyzer.py` 100%, `logging_setup.py` 100%, `models.py` 100%, `cli.py` 98% (line 79), `api.py` 86% (lines 24–27, 78–81, 84 — error branches requiring live multipart upload), `storage/base.py` 75% (abstract method bodies at lines 11, 16, 21 — unreachable by design). The one warning is a Python 3.13 deprecation notice on `imghdr`; it does not affect correctness on Python 3.11.

6.2 Notable Tests

- (i) `test_storage.py::test_repository_save_and_list_all_roundtrip`
Happy-path end-to-end test. Creates a real `aiosqlite` database in a `tmp_path`, saves an `AnalysisRecord` with one ingredient row, reads it back with `list_all()`, and asserts field-by-field equality. This test exercises the full storage boundary: SQLAlchemy DDL, ORM insert, commit, select, and `_to_model` deserialization.
- (ii) `test_concurrency.py::test_fetch_nutrition_parallel_skips_failed_lookup`
Concurrency test. Monkeypatches `USDAProvider` with a fake that raises `RuntimeError` for an ingredient named “bad ingredient”. Asserts that `fetch_nutrition_parallel` returns a dict containing “rice” and “chicken” but not “bad ingredient”, confirming `asyncio.gather` degrades gracefully when one task raises.
- (iii) `test_failure_injection.py::test_parallel_...calls_fail`
Error-path test that found a real bug. Passes three failing ingredients and asserts the result is an empty dict (not a raised exception). During development, this test failed because our first implementation used `return_exceptions=True` but also had a `try/except` inside `_fetch_one`—the two patterns conflicted. The test forced us to choose: keep `try/except` in `_fetch_one` (our current design) and use `return_exceptions=False` on `gather`.

7. Deployment & Reproducibility

7.1 Docker image

The Dockerfile uses a **single-stage** build from `python:3.12-slim`:

```
# Build
docker build -t team-sigmoid .

# Run API server
docker run --env-file .env -p 8000:8000 team-sigmoid

# Run CLI inside container
docker run --env-file .env team-sigmoid \
    python -m foodanalyzer analyze data/rice_chicken.png

# Run tests inside container
docker run --env-file .env team-sigmoid \
    pytest tests/test_ai_smoke.py -v
```

We chose `python:3.12-slim` (Debian-slim base, ~130 MB) over Alpine because several dependencies (SQLAlchemy C extensions, numpy) require `glibc` which Alpine does not ship, and the multi-stage complexity to add `musl` equivalents was not justified at this project scale.

tabularx booktabs

7.2 Configuration

All configuration is read from environment variables (or `.env`):

Variable	Default	If missing
LLM_PROVIDER	openai	Uses OpenAI path.
LLM_MODEL	gpt-4o-mini	Provider default.
OPENAI_API_KEY	empty	Provider call fails; tenacity retries 3× then raises.
OPENAI_BASE_URL	not set	Uses official OpenAI endpoint.
ANTHROPIC_API_KEY	empty	As above if LLM_PROVIDER=anthropic.
GOOGLE_API_KEY	empty	As above if LLM_PROVIDER=google.
USDA_API_KEY	empty	USDA calls fail; pipeline degrades gracefully.
DATABASE_URL	sqlite+aiosqlite:///...	Creates local SQLite file at project root.
CACHE_TTL_SECONDS	86400	24 h TTL.
NUTRITION_CONCURRENCY_LIMIT	10	10 parallel USDA requests.
HOST / PORT	0.0.0.0 / 8000	API binds to all interfaces on port 8000.
LOG_LEVEL	INFO	INFO-level logging.
MAX_IMAGE_BYTES	5242880	5 MB upload limit.

8. Limitations & Future Work

- **In-memory cache is process-local and not persistent.** If the API server restarts, all cached USDA lookups are lost and the next 24 h of cold requests pay full network cost. A Redis or SQLite-backed cache would fix this.
- **No retry on USDA calls.** The tenacity policy covers `ai_service` but not `pipeline._fetch_one`. A transient USDA 500 is silently logged and the ingredient is dropped from the result rather than retried. Adding a per-call retry with a short budget (2 attempts, 0.5 s backoff) would improve completeness.
- **Single-process, single-database.** The SQLite + aiosqlite stack is not safe for multiple concurrent writers. Moving to PostgreSQL with asyncpg would be required for any multi-replica deployment.
- **Benchmark uses fake provider.** The 5.31× speedup is measured with `time.sleep(0.05)` not real network I/O. Real USDA calls have variable latency; actual speedup in production depends on network conditions and would likely be higher since real RTT dominates thread-scheduling overhead.
- **imghdr deprecation.** `src/api.py` uses the `imghdr` module for magic-byte validation, which is deprecated in Python 3.11 and removed in 3.13. A production hardening step would replace it with `python-magic` or manual byte inspection.
- **Worst design decision:** storing `ingredients_json` as a raw JSON text column rather than a properly normalised `ingredients` table. This makes it impossible to query “which analyses contain chicken” without a full table scan and JSON parsing in application code.

9. Tools & Acknowledgements

Throughout the project we used Claude as a coding assistant in the same way a developer would use it in industry: for drafting boilerplate, suggesting library usage, and helping diagnose bugs. All AI-generated suggestions were reviewed, tested, and adapted by the team before being committed.

In particular, the assistant helped us explore tenacity retry configuration, `asyncio.gather` error-handling patterns, and diagnose the `pydantic-settings` vs `os.getenv` environment-loading incompatibility.

Every team member understands the full codebase and is able to explain and defend every line of code during the oral defense. “The AI wrote it” is not an answer any of us will use.

References

- [1] Anthropic API documentation, <https://docs.anthropic.com/>
- [2] tenacity library documentation, <https://tenacity.readthedocs.io/>
- [3] FastAPI documentation, <https://fastapi.tiangolo.com/>
- [4] pydantic-settings documentation, https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [5] USDA FoodData Central API, <https://fdc.nal.usda.gov/api-guide.html>
- [6] SQLAlchemy async documentation, <https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html>
- [7] Python asyncio documentation, <https://docs.python.org/3/library/asyncio.html>

A. Appendix — Reproducing the benchmark

```
# Clone and set up
git clone https://github.com/thedarkstringg/team-sigmoid
cd team-sigmoid
git checkout v1.0-final
cp .env.example .env          # no real keys needed for offline bench

python -m venv .venv
# Windows:
.venv\Scripts\activate
# Linux/macOS:
source .venv/bin/activate

pip install -r requirements.txt

# Run the offline benchmark (no API keys required)
set PYTHONPATH=%CD%          # Windows
# export PYTHONPATH=$(pwd)    # Linux/macOS
python scripts/bench.py
```

Expected output (approximate, hardware-dependent):

Sequential vs concurrent nutrition lookup benchmark

mode	items	duration_seconds
sequential	12	0.608
concurrent	12	0.115

speedup: 5.31x		

End of report — thank you for reading.